

Politecnico di Milano
School of Industrial and Information Engineering

Jan Wangrat

Intelligent Job Management

Final Report
COMPUTER SCIENCE AND ENGINEERING

Supervisor:
Prof. Danilo Ardagna
Politecnico di Milano
with Federica Filippini, Ph.D. Candidate

Milano, May 20, 2026

Abstract

Intelligent Job Management (IJM) is a scheduler for shared GPU clusters that combines profile-aware placement, deadline-driven preemption, and cross-version checkpoint resume. It extends the ANDREAS project (Politecnico di Milano) from a single-host prototype to a distributed control plane: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, an external GPUspb optimiser handles standard-run placement, and training jobs migrate between heterogeneous GPU generations (A40, QuadroP600) without losing progress. The report describes the deployment topology, the four-phase scheduling pipeline, the configuration surface, and two end-to-end scenarios that exercise the system on a two-node cluster.

Keywords

GPU scheduling, deep learning, job management, profile-aware optimisation, distributed systems, FastAPI, PostgreSQL, Docker

Contents

1. Introduction	2
2. System Architecture	3
2.1. Deployment topology	3
2.2. Scheduling pipeline	3
2.3. State machine	5
2.4. Concurrency invariants	6
2.5. Cross-node checkpoints and image variants	6
3. Configuration and Job Submission	9
3.1. Cluster configuration	9
3.1.1. Nodes (<code>nodes_config.json</code>)	9
3.1.2. Energy costs (<code>gpu_energy_costs.json</code>)	10
3.2. Job submission	10
3.2.1. Request body	10
3.2.2. Response (201 Created)	11
3.3. Full API overview	11
3.4. Runtime environment	12
4. End-to-End Scenarios	14
4.1. Objective and cost model	14
4.1.1. Cluster	14
4.1.2. Proxy vs. true objective	14
4.2. Available job types	15
4.3. Scenario 1 — single type	15
4.3.1. Decision timeline	17
4.3.2. URGENT plan comparison — true objective	17
4.3.3. Why URGENT lands on the slowest 2-GPU bundle	17
4.4. Scenario 2 — two types, forced 2-GPU placement	19
4.4.1. Settled steady state (before URGENT)	20
4.4.2. URGENT placement and migrations	20
4.4.3. Measured cascade	20
4.5. Implementation notes	20
4.5.1. Slot-tracker race on quick stop/run pairs	20
4.5.2. Worker progress-lag and stop-drain stalls	21
4.5.3. Profile-claim lifecycle thrash	21
4.5.4. Reaper-wake gap after migrate-preempts	21
4.5.5. Cross-version checkpoint compatibility	21
4.6. Reproducibility	22
Bibliography	23

1. Introduction

Modern deep-learning research depends on shared GPU clusters in which training jobs compete for a small number of expensive accelerators. Operators are expected to place each job on the hardware that meets its deadline at the lowest cost, despite per-GPU prices that vary by more than an order of magnitude between generations. Manual scheduling quickly becomes the bottleneck, and off-the-shelf cluster managers (Kubernetes, plain SLURM) treat GPU bundles as interchangeable and have no notion of per-job energy budgets or deadline-aware preemption.

The ANDREAS project [AF⁺21] at Politecnico di Milano demonstrated that a cost-aware optimiser, fed with profile measurements of each job type on each hardware bundle, can plan placements that beat the typical operator’s intuition. ANDREAS, however, is a single-host prototype: it runs every container on the machine that hosts the API, has no remote worker, and assumes one homogeneous GPU pool. Real laboratories — the two-node MIM UW / Polimi cluster used in this work being a representative example — run jobs across nodes with different driver generations, must resume preempted training across CUDA versions, and want to consume a standalone optimiser as an HTTP service rather than ship it inside the scheduler.

This report describes Intelligent Job Management (IJM), a scheduler that extends the ANDREAS model to a distributed cluster. The control plane is a single FastAPI service that hosts the profiling scheduler, the optimiser client, and the per-node slot semaphores. Each GPU node runs a lightweight worker that owns the local Docker daemon. The two are glued together by plain HTTP over SSH tunnels, so the same code runs on a laptop talking to a remote cluster and on a single-host development machine. IJM integrates the GPUspb optimiser as the external decision maker, enforces a *profile-always* policy so no standard run ever starts on an unmeasured bundle, and saves checkpoints in a format that loads across torch versions, allowing a job to migrate between an A40 node and a legacy P600 node mid-training.

We validate the design with two end-to-end scenarios on the two-node cluster. Scenario 1 submits five `cnn_big` jobs to probe the profile sweep and the deadline-driven URGENT migration sequence. Scenario 2 mixes two job types and forces the optimiser to place an urgent `cnn_big` on a 2-GPU bundle to meet a tight deadline while preempting two slack `lstm-small` patients. Both scenarios complete without manual intervention, with all jobs reaching SUCCEEDED.

Chapter 2 describes the deployment topology, the scheduling pipeline, the state machine, and the invariants that protect the control plane from concurrency hazards. Chapter 3 documents the cluster and job configuration files, the submission API, and the runtime knobs. Chapter 4 presents the two scenarios, the cost model used by the optimiser, and the implementation notes accumulated while bringing the scenarios to a reproducible state.

2. System Architecture

This section presents IJM’s deployment topology, the four-phase scheduling pipeline that drives every dispatch decision, the job state machine, and the concurrency invariants that keep the control plane consistent under racing API calls and worker notifications.

2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* — which also hosts the `ProfilingScheduler`, the `JobDispatcher`, the `NodeSlots` semaphores, and the optimiser client — runs on the operator’s machine. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes.

Communication is plain HTTP over SSH tunnels. Every worker is reached from the API at a stable `http://localhost:800x` alias regardless of its physical location, so the same scheduler code drives a laptop talking to a remote cluster and a single-host development setup. An optional GPUspb *optimiser* HTTP service is reached the same way.

Single-process `docker compose` on the operator’s machine is supported for development but is not the deployment target: any tradeoff between local-dev simplicity and distributed correctness resolves toward the distributed topology.

2.2 Scheduling pipeline

A single scheduling pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision. It is invoked whenever a job is submitted, a profiling run finishes, or a slot is freed. A 60-minute watchdog (`_queue_watcher`) serves as a safety net for missed notifications, not as the main wake source. Wake-ups are coalesced through a single `asyncio.Event` and rate-limited to at most one pass every five seconds, so the cluster has time to apply the previous plan before the next one is computed.

The pass walks four phases under a single `schedule_lock`:

1. **Phase 0 — sweep stale state.** Sweep stale profile claims and reset stuck `QUEUED` rows whose `assigned_node` was set ≥ 30 s ago without the worker advancing them.
2. **Phase 1b — optimiser pass.** Call the optimiser outside the lock (to accommodate its 30 s HTTP timeout) and apply its assignments through a conditional `UPDATE ...WHERE assigned_node IS NULL`, collecting the optimiser’s preempt list.
3. **Greedy fallback.** For `QUEUED` rows the optimiser did not place — typically rows that still owe a profile run — the `ProfilingScheduler` picks one profile configuration or a profiled standard slot.
4. **Phase 1c — profile-preempt.** If a row still has no slot and still owes a profile, find the cheapest set of low-priority `RUNNING` jobs whose eviction would free enough GPUs for the next unprofiled configuration (the *profile-always* policy).

Outside the lock the pass spawns one task per preempt and one per dispatch. Each dispatch goes through `JobDispatcher.dispatch_with_slot`, which acquires the destination’s `NodeSlots` permit before sending `POST /jobs/{id}/run` to the right worker. Permits are released when the worker emits a `NOTIFY ijm_slot_freed` that carries the freed `(node_id, n_gpus)` pair.

Current Prototype Architecture (distributed)

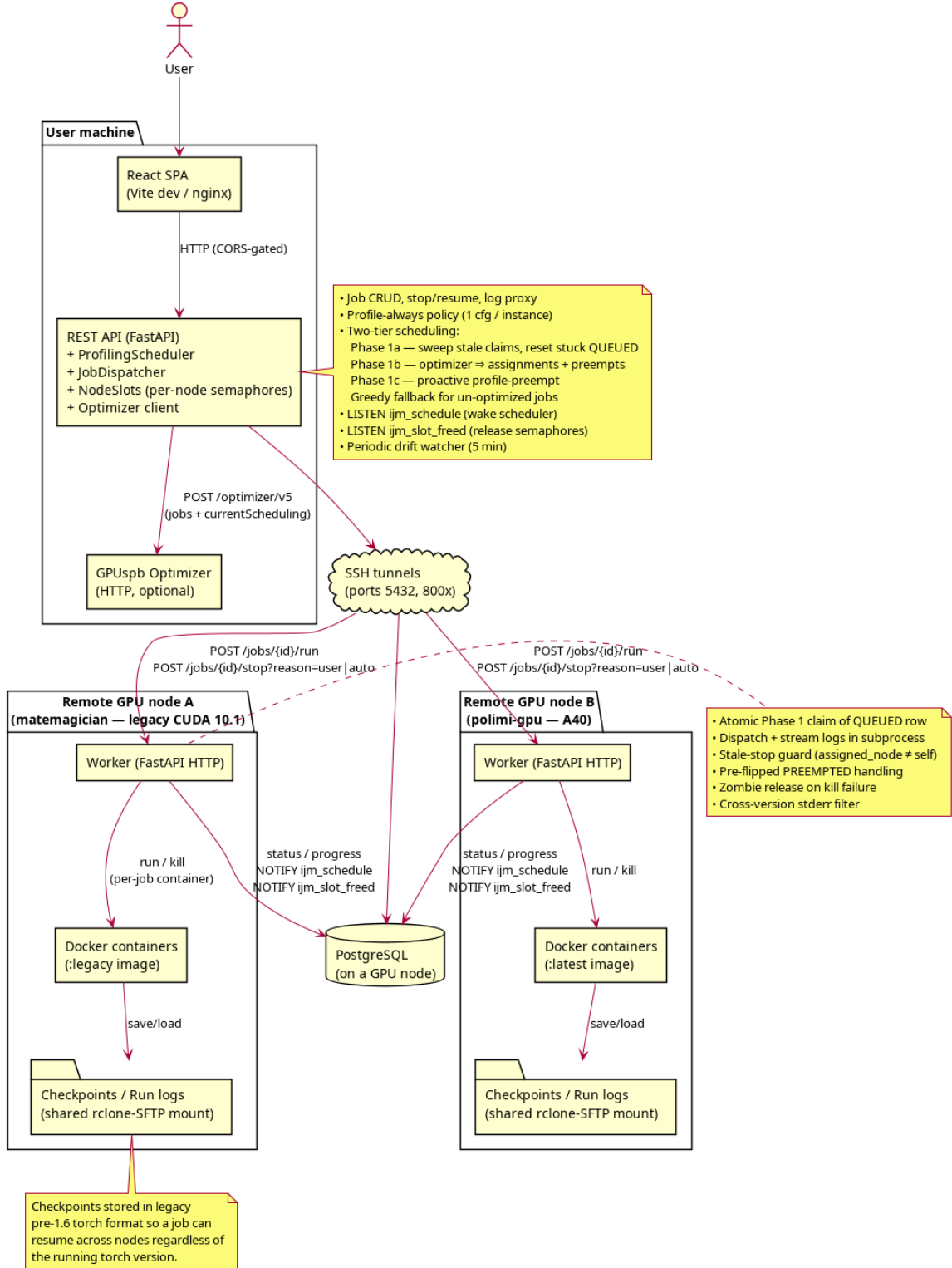


Figure 2.1: Distributed prototype. The SPA, API, and optimiser run on the operator’s machine; workers and Docker run on each GPU node; PostgreSQL runs on a GPU node. A shared rclone-over-SFTP mount makes checkpoints written by one node visible from the other.

Figure 2.2 shows the submission path, Figure 2.3 the cross-instance profile cycle, Figure 2.4 an optimiser-driven preempt and migrate round, and Figure 2.5 the standard run.



Figure 2.2: Submission and scheduling. The API writes the row and notifies the scheduler; placement is decided by the next pass. The optimiser call sits outside the schedule lock so its 30 s HTTP timeout cannot block other passes.

2.3 State machine

Table 2.1 lists every transition a job row can take. All transitions are driven by atomic conditional UPDATES, so concurrent stop, dispatch, and completion paths cannot clobber each other’s work.

From	To	Trigger
QUEUED	PROFILING / RUNNING	Worker Phase 1 atomic claim
PROFILING	QUEUED	Profile run finished; <code>is_profiling_run</code> flips to false
RUNNING	SUCCEEDED / FAILED	Container exit
RUNNING	PREEMPTED	User <code>POST /jobs/{id}/stop?reason=user</code>
RUNNING	QUEUED	Scheduler preempt (<code>reason=auto</code>): clears <code>assigned_node</code> for the next pass
PREEMPTED / FAILED	QUEUED	<code>POST /jobs/{id}/resume</code>

Table 2.1: Job state transitions.

Two distinct stop reasons separate *user-initiated* preemption from *scheduler-initiated* preemption. A user stop is sticky: PREEMPTED stays put until the user explicitly resumes. A scheduler stop returns the row to QUEUED with a cleared assignment, so the next pass can

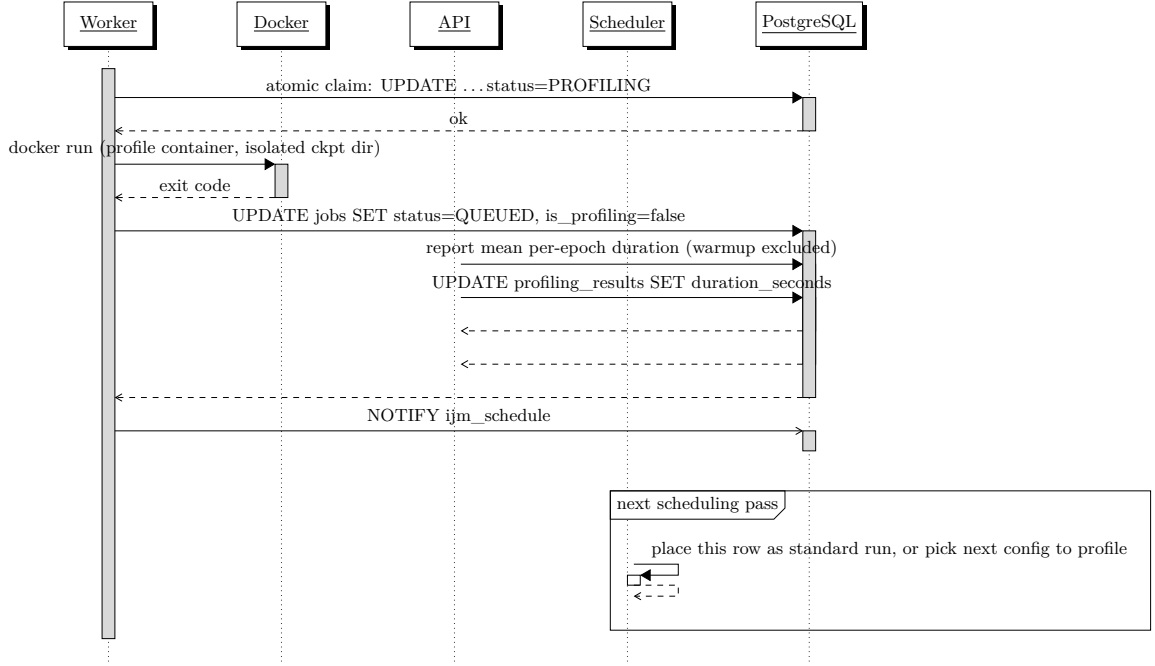


Figure 2.3: Profiling cycle. Profile measurements are keyed by the (job-type, GPU-config) pair and shared across submissions of the same type. Each instance contributes one profile run by default; the per-instance claim row guards against duplicate measurements.

place it elsewhere automatically. The worker enforces this distinction and ignores stale `/stop` calls aimed at a row that has since been re-assigned to another node.

2.4 Concurrency invariants

The control plane relies on three invariants, each defended in code.

No GPU oversubscription. Every dispatch goes through `NodeSlots.acquire(node_id, n_gpus)`. On startup the semaphores are reconciled from the database; rows in `RUNNING` or `PROFILING`, and `QUEUED` rows with an assigned node, pre-acquire their permits. A periodic drift watcher recomputes the authoritative count at the `IJM_DRIFT_HEARTBEAT_S` cadence (default 15 s) and corrects any divergence.

No duplicate containers per row. Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck `QUEUED` rows cancels any live dispatch task instead of double-releasing its permit.

Profiling is sacred. Auto-preempt (`_preempt_and_release`) refuses to act on `is_profiling_run=TRUE`, and profile-preempt (Phase 1c) filters victims by `status='RUNNING'` only. Only an explicit user stop can cancel an in-flight measurement.

2.5 Cross-node checkpoints and image variants

Two GPU generations coexist. The `matemagician` node is pinned to driver 418.x (CUDA 10.1 max), so it runs a separate `:legacy` image built on PyTorch 1.5.1 and Python 3.7. The remaining nodes run the default `:latest` image on PyTorch 2.6 and CUDA 12.4. Both images write checkpoints in the legacy pre-1.6 `torch.save` format, and both load with `strict=False`, so a job can resume across nodes regardless of which torch version produced the file.

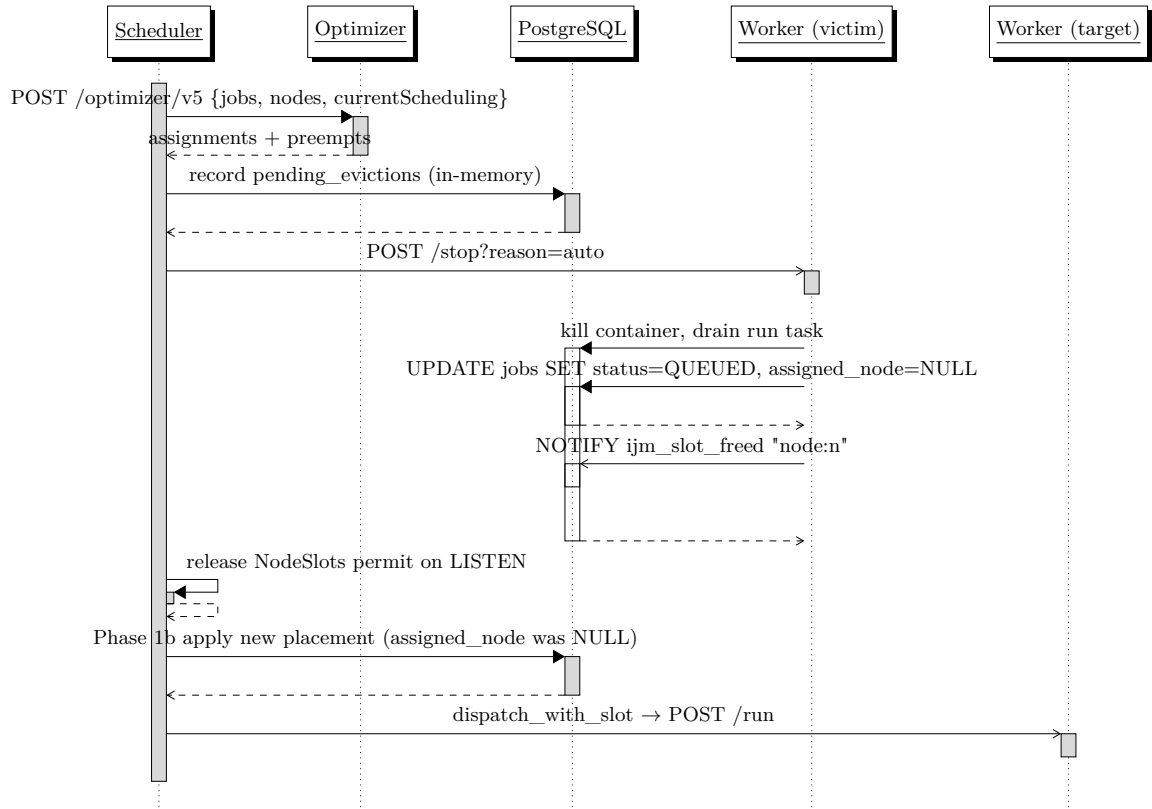


Figure 2.4: Optimiser-driven preempt and migrate. The API trusts the GPUspb plan: any job in `currentScheduling` that the optimiser drops or moves is preempted with `reason=auto`, which clears its assignment so Phase 1b can re-apply the new placement on the next pass.

The shared `data/` directory is mounted on every node via `rcclone` over SFTP. It contains:

- `data/checkpoints/{job_id}/` — per-job checkpoints, mounted into the container at `/checkpoints`;
- `data/runs/{job_id}/` — captured stdout (`output.log`) and TensorBoard artifacts, mounted at `/runs`;
- `data/shared/data/` — read-only dataset cache (MNIST, CIFAR-10), mounted at `/runs/data`.

Two worker-side environment variables let the same `nodes_config.json` drive heterogeneous nodes. `WORKER_GPU_MODE` selects how Docker exposes GPUs to the training container: `runtime` for the standard NVIDIA runtime, `cdi` for the rootless daemon on `polimi-gpu`, and `none` for a CPU-only sandbox. `IMAGE_TAG_OVERRIDE` rewrites the image tag, which is how `matemagician` swaps `:latest` for `:legacy`.

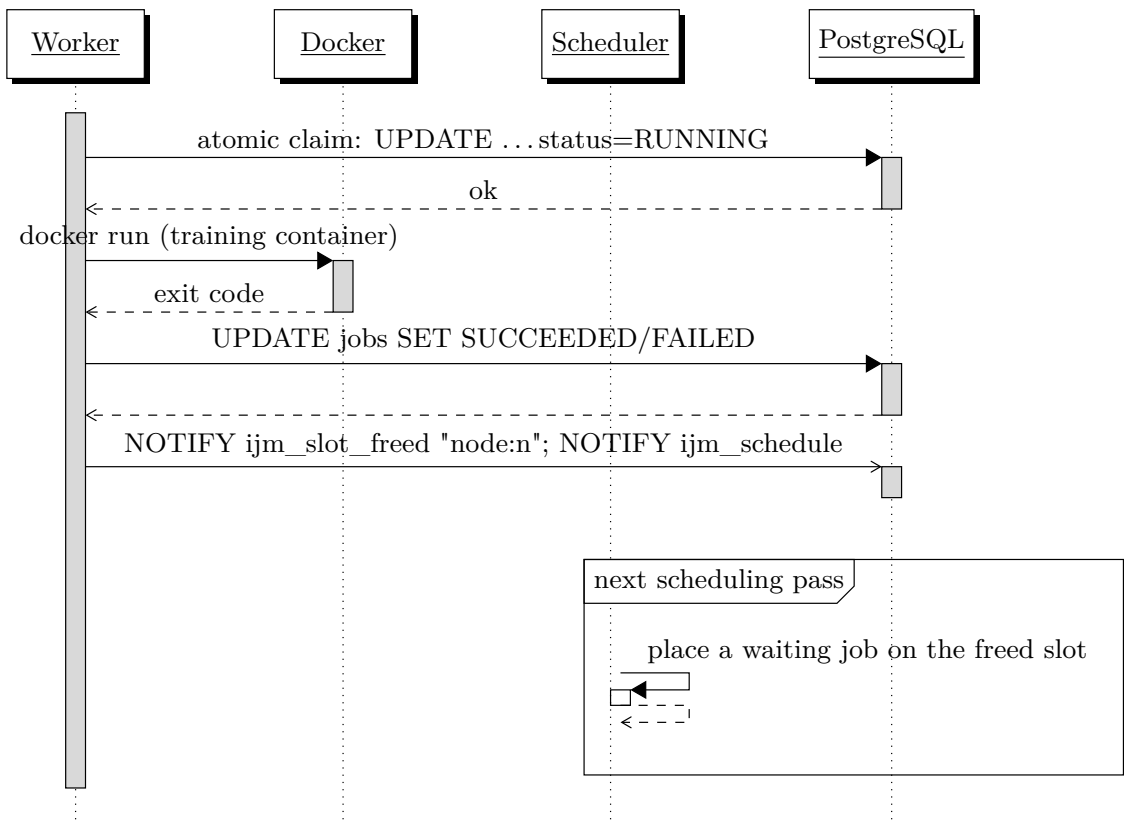


Figure 2.5: Standard run. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops or duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires `ijm_slot_freed` and `ijm_schedule` in the same transaction.

3. Configuration and Job Submission

IJM is configured at three layers: a pair of JSON files that describe the cluster, an HTTP submission API for jobs, and a handful of runtime environment variables. This section documents each in turn.

3.1 Cluster configuration

The cluster topology is loaded at startup from two JSON files, whose paths are configurable via environment variables.

3.1.1 Nodes (`nodes_config.json`)

Each entry describes a compute node and its GPU resources.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.06,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.55,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

Field	Type	Description
id	string	Unique node identifier
isForProfiling	bool	If true , the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it
workerUrl	string null	HTTP base URL of the worker, typically reached through an SSH tunnel. When null , dispatch falls through to the in-process JobRunner (development only)
cost	float	Idle cost of the node (\$/h), used by the optimiser
resources	array	GPU resources attached to the node
resources[].gpu_type	string	GPU model name (e.g. A40, L40S, QuadroP600)
resources[].gpu_count	int	Number of GPUs of this type

Table 3.1: Node configuration fields.

GPU type and count are declared explicitly rather than auto-detected. This admits mixed-GPU nodes that ANDREAS did not support (e.g. a node with both A40 and L40S GPUs), and it lets the operator declare nodes that are not physically attached to the API host — the API only ever talks to workers over HTTP.

3.1.2 Energy costs (gpu_energy_costs.json)

A lookup table that maps a GPU type and bundle size to an hourly energy cost (EUR/h). The scheduler consults this table when scoring candidate placements.

```
{
  "A40": { "1": 0.15, "2": 0.28, "4": 0.54 },
  "L40S": { "1": 0.18, "2": 0.34, "4": 0.65 }
}
```

3.2 Job submission

Endpoint: POST /jobs

3.2.1 Request body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

Field	Type	Required	Description
job_id	string	yes	Reusable job type identifier; profile measurements are correlated across submissions sharing the same <code>job_id</code>
dockerImage	string	yes	Docker image to execute
scriptPath	string	no	Path to the training script (overrides Docker CMD with <code>python3 <scriptPath></code>)
directoryToMount	string	no	Host directory mounted into the container at <code>/workspace</code>
command	string[]	no	Command and arguments (defaults to the image entrypoint; overridden by <code>scriptPath</code>)
Priority	int 1-5	no	Job priority (default 3)
deadline	datetime	no	Job deadline (ISO 8601)
batchSize	int	no	Batch size passed as the <code>BATCH_SIZE</code> env var
profilingEpochsNo	int ≥ 1	no	Epochs per profiling run (default 3)
epochsTotal	int ≥ 1	no	Total training epochs (default 20)

Table 3.2: Job creation request fields.

Each submission receives a unique UUID instance ID (`id`) for lifecycle tracking, separate from the `job_id` type identifier. When `scriptPath` is provided the container runs `python3 <scriptPath>`; otherwise the image's default CMD is used. Unlike ANDREAS, `Priority` and `deadline` are optional, and an additional `command` field gives flexibility for non-standard workloads.

3.2.2 Response (201 Created)

The response carries the full job object, including the scheduler's initial decision: assigned node, GPU configuration, and whether the first run will be a profile or a standard execution.

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "node-mixed-prof",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

3.3 Full API overview

Table 3.3 lists every endpoint. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

Method	Path	Description
GET	/	Health check endpoint
GET	/health	Detailed health check — pings DB and checks job runner
GET	/nodes	List all cluster nodes with their current status
GET	/gpu-costs	Hourly GPU energy costs used by the scheduler for optimisation
GET	/configurations	List all valid hardware configurations in the cluster
POST	/jobs	Create a new job
GET	/jobs	List all jobs ordered by creation time (paginated via <code>limit/offset</code>)
GET	/jobs/{id}	Get a specific job by ID
POST	/jobs/{id}/stop	Request job to be stopped (user-driven; sticky PRE-EMPTED state)
POST	/jobs/{id}/resume	Request job to be resumed
DELETE	/jobs/{id}	Delete a job, killing any running container first
DELETE	/jobs	Delete all jobs and their profiling results
GET	/jobs/{id}/logs	Return the output log for a job by proxying to the worker that owns it
GET	/profiling-results/{id}	All profiling results for a job, ordered by duration (fastest first)
GET	/admin/slots	NodeSlots snapshot: in-memory used vs. DB-authoritative count
POST	/admin/reconcile-slots	Force NodeSlots to match DB-authoritative state (idempotent)
GET	/admin/dispatch-tasks	List in-flight <code>_dispatch_when_slot_free</code> tasks

Table 3.3: API endpoints.

Three architectural choices set the API apart from ANDREAS. First, scheduling, profiling, and dispatch are consolidated into a single FastAPI service, in contrast to ANDREAS’s split between a Java Jobs Manager and a C++ Jobs Optimiser. Second, execution lives in per-node *workers* — also FastAPI — that the API reaches over HTTP through SSH tunnels; the external GPUSpb optimiser is reached the same way and owns every standard-run placement decision. Third, IJM exposes explicit `stop`, `resume`, `logs`, and `delete` endpoints, together with `/profiling-results` and `/configurations`, so the user has direct visibility into the scheduler’s state.

3.4 Runtime environment

Table 3.4 lists the runtime knobs that influence the scheduler. Unless noted otherwise, every variable is read by the API process at startup.

Variable	Default	Description
DATABASE_URL	postgresql://.../ijm	PostgreSQL DSN, reached via SSH tunnel
OPTIMIZER_URL	unset	The GPUspb HTTP service used for standard-run placement
OPTIMIZER_METHOD	RG	GPUspb solver method passed in the request body
OPTIMIZER_VERBOSE	0	Per-call log verbosity (0–3)
PROFILING_CONFIGS_PER_JOB	2	How many distinct GPU configurations a single instance profiles before falling through to a standard run
EXECUTOR	docker	Executor backend for the in-process JobRunner: <code>docker</code> or <code>mock-slurm</code> . Workers always use the Docker CLI
IJM_DRIFT_HEARTBEAT_S	15	Period of the slot-tracker drift heartbeat that reconciles <code>mem_used</code> against <code>db_used</code> per node
WORKER_GPU_MODE	runtime	Worker-side; how Docker exposes GPUs to training containers (<code>runtime</code> / <code>cdi</code> / <code>none</code>)
IMAGE_TAG_OVERRIDE	unset	Worker-side; rewrites <code>:latest</code> to the given tag. Used on <code>matemagician</code> to pick the CUDA-10.1-compatible <code>:legacy</code> image
HOST_ROOT, HOST_PROJECT_ROOT	/host, \${PWD}/..	Resolve container-internal vs. host paths for Docker bind mounts

Table 3.4: Key environment variables.

4. End-to-End Scenarios

This section presents the cost model and the two end-to-end scenarios used to validate IJM on the two-node cluster. Scenario 1 exercises the single-type profile sweep and the deadline-driven URGENT migration. Scenario 2 adds a second job type and forces a 2-GPU placement to meet a tight deadline. The last subsection collects implementation notes accumulated while bringing the scenarios to a reproducible state.

4.1 Objective and cost model

The scheduler minimises

$$\text{TOTALCOST} = \sum_j \text{GPUcost}(j) + \sum_j \text{TardWeight}(j) \cdot \max(0, \text{finish}(j) - \text{deadline}(j)),$$

where **GPUcost** is wall-clock $\times$ hourly rate (Table 4.1, left) and **TardWeight** scales with priority (right). Slack deadlines favour the cheapest bundle; deadline pressure shifts urgent jobs onto the fastest available.

GPU bundle	\$/hour	Priority	TardWeight (\$/sec)
QuadroP600 $\times$ 1	0.030	1	0.000254
QuadroP600 $\times$ 2	0.055	4	0.000396
A40 $\times$ 1	0.300	5	0.000444
A40 $\times$ 2	0.550		

Table 4.1: Bundle hourly rates and per-priority tardiness weights. Two-GPU bundles include a small co-location discount.

4.1.1 Cluster

The cluster used throughout this section consists of two profiling-capable nodes and four slots total:

Node	GPUs	Bundle hourly cost
matemagician(n1)	2 $\times$ QuadroP600	0.055
polimi-gpu(n2)	2 $\times$ A40	0.550

For each new job type the profile sweep exercises all four (**node**, **GPU-count**) cells (**configs_per_job**=2, four instances).

4.1.2 Proxy vs. true objective

The cost above is the modelled objective. The C++ optimiser’s per-decision proxy (**proxy_function_min**/**proxy_function_max** in GPUspb) is a cheaper surrogate.

In the minimisation form used by **method=RG**, tardiness is evaluated at the *next-event horizon* **SIM_TIME=CURRENT_TIME+FIRST_FINISH_TIME** rather than at each job’s predicted completion. A placement on slow hardware therefore contributes zero in-proxy tardiness as long as the deadline is still in the future at that horizon.

Combined with the 100 $\times$ multiplier the proxy applies to the worst-case tardiness of jobs left *unscheduled* in a candidate plan, this explains the URGENT-on-P600 $\times$ 2 landing observed

in our runs. With the proxy reading zero tardiness for every URGENT placement, the only term that distinguishes the options is the worst-case tardiness of jobs the plan would have to drop. Any plan that keeps every job scheduled wins over a plan that does not.

The proxy never names “preemption”: it just penalises “unscheduled in this plan”. The `proxy_function_max` variant (`method=PR`) uses per-job `sch.get_selectedTime()` instead and is deadline-aware, but it is not the default. We keep `method=RG` throughout to mirror the reference deployment. The behaviour is present on every actively-maintained upstream GPUspb branch as of 2026-01.

4.2 Available job types

Type	Architecture	Dataset
<code>lstm-small</code>	LSTM 1-layer, 128 hidden	MNIST (sequence)
<code>lstm-big</code>	LSTM 3-layer, 256 hidden, dropout 0.2	MNIST (sequence)
<code>convnet</code>	3-layer CNN + BN	CIFAR-10
<code>efficientnet</code>	MBCnv EfficientNet	CIFAR-10
<code>cnn_big</code>	10-block CNN, $3 \rightarrow \dots \rightarrow 512$ channels	synthetic $128 \times 128 \times 3$

Table 4.2: Training containers under `runtime/`. All share the same checkpoint contract (legacy pre-1.6 `torch.save`, loaded with `strict=False`), so any job can resume across nodes regardless of the destination’s torch version.

Type	A40×1 [s/ep]	A40×2 [s/ep]	QuadroP600×1 [s/ep]	QuadroP600×2 [s/ep]
<code>lstm-small</code>	3.56	4.81	5.47	10.93
<code>cnn_big</code>	4.171	2.854	46.141	27.504

Table 4.3: Steady-state per-epoch wall-clock from `profiling_results` rows captured on 2026-05-20 (mean of post-warmup epochs; the per-type sweep runs were separate). Bold marks 2-GPU bundles that beat their 1-GPU counterpart on wall-clock. `cnn_big` wins $1.46\times$ on A40×2 and $1.68\times$ on P600×2; it is the only type used by Scenario 1. Only profiled types are listed.

Type	A40×1 [\$/ep]	A40×2 [\$/ep]	P600×1 [\$/ep]	P600×2 [\$/ep]
<code>lstm-small</code>	0.000297	0.000735	0.0000456	0.000167
<code>cnn_big</code>	0.000348	0.000436	0.000385	0.000420

Table 4.4: GPUcost per epoch, computed as $\text{per-epoch[s]} \times \text{hourly-rate (Table 4.1)} / 3600$. Multiply by `epochsTotal` to obtain the optimiser’s energy term for a plan that runs the job to completion on the listed bundle.

4.3 Scenario 1 — single type

We submit five `cnn_big` jobs (Table 4.5): two tight prio-4 patients (75 ep each, +30-min deadline), two slack low-priority patients (50 and 25 ep, +2-h deadline), and one URGENT (200 ep, +10-min deadline) injected after the patients settle. Staggered priorities and deadlines give each job a unique preferred slot, so the resulting placement is reproducible. The +30-min patient deadline is well past what any A40 placement needs, so a plan that preempts a patient to make room for URGENT carries near-zero real cost — yet, as we shall see, the proxy never proposes one. The captured run was produced with the scenario script’s default settings

(JOB_TYPE=cnn_big bash infra/e2e_scenario.sh); the values in Table 4.5 are the ones that produced the timelines and decision math below.

Label	Type	Priority	Deadline	Epochs	Submitted at
JOB1	cnn_big	4	+30 min	75	$t = 0$ s
JOB2	cnn_big	4	+30 min	75	$t = 7$ s
JOB3	cnn_big	2	+2 h	50	$t = 14$ s
JOB4	cnn_big	1	+2 h	25	$t = 19$ s
URGENT*	cnn_big	5	+10 min	200	$t=7.5$ min

Table 4.5: Scenario 1 submissions. Two tight prio-4 patients ($A40 \times 1$ candidates), two slack low-priority patients ($P600 \times 1$ candidates), and one URGENT after settlement.

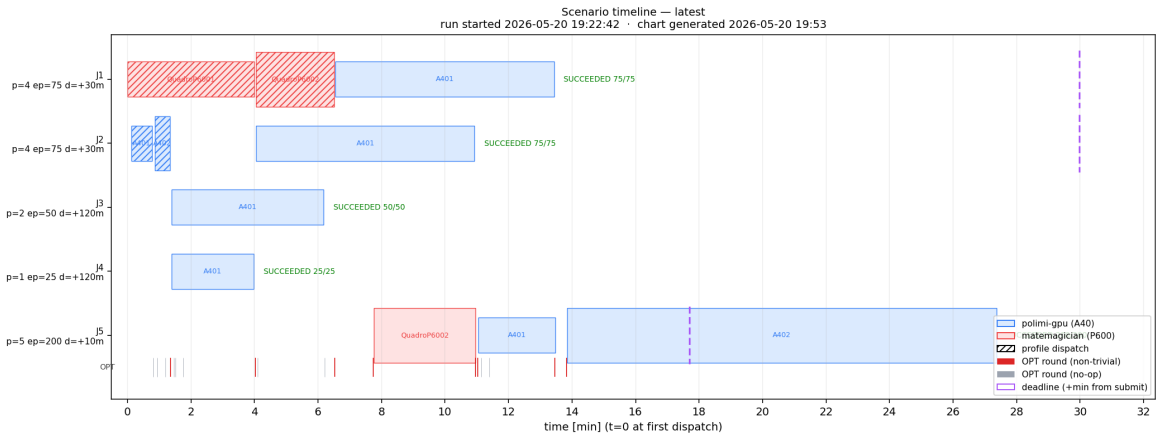


Figure 4.1: Scenario 1 timeline (run 2026-05-20, $t=0$ at 19:22:42 CEST, patient deadlines = +30 min). Upstream GPUsp method=RG. Solid blue and red bars are **polimi-gpu** A40 and **matemagician** P600 runtime; hatched bars are profile dispatches. Five SUCCEEDED, zero FAILED. JOB1 carries the **matemagician** P600 sweep; JOB2 carries the **polimi-gpu** A40 sweep; JOB3 and JOB4 skip profiling via the atomic claim and start standard runs first. URGENT (submit $t=7.5$ min, deadline = +10 min) lands on **matemagician** $P600 \times 2$ even though both P600 slots are free, runs there until JOB2 finishes naturally at $t \approx 11$ min freeing an A40 slot, migrates to $A40 \times 1$, then upgrades to $A40 \times 2$ when JOB1 finishes at $t \approx 13.5$ min, and finally completes at $t \approx 27.5$ min — about 10 min past its deadline (purple dashed mark). The migration sequence is analysed in §4.3.3.

Table 4.6 gives the per-bundle measurements that landed in **profiling_results** during the sweep. Two features drive the rest of the scenario. First, $P600 \times 1$ is the cheapest 50-epoch slot for slack jobs that can absorb a 38-minute wall. Second, only $A40 \times 2$ (200 ep ≈ 9.5 min) fits a 10-minute deadline for a 200-epoch URGENT.

Bundle	per-epoch [s]	GPUcost/ep [\$]	50-ep [\$]	200-ep [\$]	Best for
$A40 \times 1$	4.171	0.000348	0.0174	0.0696	medium-deadline cnn_big
$A40 \times 2$	2.854	0.000436	0.0218	0.0872	≤ 10-min URGENT
QuadroP600 $\times 1$	46.141	0.000385	0.0192	0.0769	slack runs (cheapest)
QuadroP600 $\times 2$	27.504	0.000420	0.0210	0.0840	— (dominated)

Table 4.6: Per-bundle data the optimiser sees for **cnn_big** after Scenario 1’s profile sweep. Per-epoch is the steady-state mean from **profiling_results** (warmup epoch excluded). Full 200-epoch walls: $A40 \times 1 = 13.9$ min, $A40 \times 2 = 9.5$ min, $P600 \times 1 = 153.8$ min, $P600 \times 2 = 91.7$ min. Only $A40 \times 2$ fits URGENT’s 10-minute deadline; the other three overrun and pay tardiness.

4.3.1 Decision timeline

Table 4.7 lists the scheduling events visible on Figure 4.1: profile-sweep claims, standard-run dispatches, the URGENT placement, and the two URGENT migrations. Bold rows correspond to the migration arrows on the chart. The proxy rationale that drove the URGENT placement and migrations is unpacked in §4.3.3.

t [min]	Event	Job(s)	Action
0.0	Sweep start	J1, J2	J1 profile P600×1 mate; J2 profile A40×1 polimi
0.6	A40×1 profile done	J2	J2 profile A40×2 polimi
1.4	A40×2 profile done	J2, J3, J4	J2/J3 standard A40×1 polimi; J4 standard A40×1 polimi
3.0	P600×1 profile done	J1	J1 profile P600×2 mate
4.0	J4 done (25/25)	J4	A40 slot frees
6.5	P600×2 profile done; J3 done (50/50)	J1	J1 standard A40×1 polimi
7.5	URGENT submit	URGENT	—
8.0	URGENT plan accepted	URGENT	dispatched P600×2 mate
11.0	J2 done (75/75); A40 slot frees	URGENT	migrate → A40×1 polimi
13.5	J1 done (75/75); 2nd A40 slot frees	URGENT	upgrade → A40×2 polimi
17.5	URGENT deadline	URGENT	still running (12 min remaining)
27.5	URGENT done (200/200)	URGENT	finishes ~10 min past deadline

Table 4.7: Scenario 1 event timeline (times read from Figure 4.1). Bold rows mark the URGENT placement and its two reactive migrations. Note that the migrations are triggered by patients finishing naturally, not by a policy decision to free A40 for URGENT — which is the bug §4.3.3 dissects.

4.3.2 URGENT plan comparison — true objective

At URGENT’s submission ($t=7.5$ min, 19:30:24 local), the true objective would order the plans by full-job ExecTime + TardCost against URGENT’s +10-minute deadline. Only A40×2 stays under the deadline at full job length; the other three options accrue priority-5 tardiness at $w_5=0.000444$ \$/s.

Plan for URGENT	ExecTime [s]	Tardy [s]	GPUcost [\$]	TardCost [\$]	TotalCost [\$]	True-cost rank
A40×2 (preempt 2)	571	0	0.0872	0.000	0.0872	1 (best)
A40×1 (preempt 1)	834	234	0.0696	0.104	0.174	2
P600×2 (preempt 2)	5 500	4 900	0.0840	2.176	2.260	3
P600×1 (preempt 1)	9 228	8 628	0.0769	3.831	3.908	4

Table 4.8: True-objective ranking of URGENT plans at submission. ExecTime = 200×per-epoch from Table 4.6; Tardy = max(ExecTime − 600, 0) against the 10-minute deadline; TardCost = Tardy × w_5 . Only A40×2 meets the deadline. This is *not* the cost the proxy used to decide; see §4.3.3.

4.3.3 Why URGENT lands on the slowest 2-GPU bundle

A naïve reading of Table 4.8 suggests A40×2 should win: it is the only plan that meets URGENT’s 10-minute deadline at full job length. The captured run instead places URGENT on `matemagician` P600×2 at $t=8.0$ min, migrates it to `polimi-gpu` A40×1 at $t=11.0$ once JOB2 finishes, and finally upgrades it to A40×2 at $t=13.5$ when JOB1 finishes. URGENT completes at $t=27.5$ min — 10 min past its deadline. The +30-minute patient deadlines are deliberately loose (a preempted patient could re-run on any bundle and still finish on time), so the “true” cost of preempting one is near zero — yet the proxy never proposes such a plan.

Three proxy mechanics combine to force this choice; we unpack them below.

1. **Horizon-myopic tardiness.** `proxy_function_min` evaluates each job’s tardiness at `SIM_TIME=CURRENT_TIME+FIRST_FINISH_TIME`, where `first_finish_time` is the soonest currently-scheduled finish. At URGENT’s arrival, the A40 patients have ~ 30 ep remaining on A40 $\times$ 1 (≈ 125 s); URGENT is 200 ep ahead on any bundle, so it cannot finish in this horizon. Its deadline (+10 min = 600 s) is still in the future at `SIM_TIME`, so *every* URGENT placement gets in-proxy tardiness = 0. Tardiness drops out of the URGENT-on-X vs. URGENT-on-Y comparison.
2. **Plans that drop a runnable job pay the $100\times$ `worstTardiCost` penalty.** The proxy has no explicit “preempt” term: for every candidate schedule it loops over jobs and charges `vmCost` + `tardiCost` for the scheduled ones and $100 \cdot \max(\text{CURRENT_TIME} + \text{MAX_EXEC_TIME} - \text{deadline}, 0) \cdot w_{\text{prio}}$ for the unscheduled ones (`sch.isEmpty()`, `scheduler.cpp` L178–189). Putting URGENT on A40 $\times$ 1 leaves one priority-4 patient unscheduled; A40 $\times$ 2 leaves both. Each evicted patient’s `maxExecTime` is its 75-epoch wall on the *worst* eligible bundle (P600 $\times$ 1, ≈ 3461 s), so even though the patient could re-run on A40 and meet its +30-minute deadline in practice, the proxy charges $100 \cdot (7.5 \text{ min} + 3461 \text{ s} - 30 \text{ min}) \cdot w_4 \approx \83 per displaced patient. A40 $\times$ 2 doubles that. In practical terms, any plan that leaves a runnable `cnn_big` patient unscheduled is two orders of magnitude more expensive than any plan that keeps everyone scheduled — even when, in reality, the patient has enough slack to be safely preempted and re-run.
3. **P600 $\times$ 2 vs. P600 $\times$ 1: the `WorstTardy` tiebreaker.** Both P600 plans keep every job scheduled (`matemagician` was empty post-sweep), so neither pays the $100\times$ unscheduled penalty. The remaining differentiator is the in-schedule `tardiCost` at horizon — 0 for both at $t=7.5$ — plus URGENT’s own `maxExecTime`-tardy, which the proxy uses when ordering equal-tardiness plans. URGENT’s worst case on P600 $\times$ 1 is 153.8 min (143.8 min late); on P600 $\times$ 2 it is 91.7 min (81.7 min late). P600 $\times$ 2 has the smaller absolute worst-case, which is enough to break the tie.

Plan for URGENT	vmCost@hzn [\$]	Unscheduled jobs	worstTardiCost [\$]	Proxy total
P600$\times$2 (chosen)	0.0019	0 (none)	0	≈ 0.03
P600 $\times$ 1	0.0010	0 (none)	0	similar order; loses on URGENT’s worst case
A40 $\times$ 1	0.0104	1 patient	≈ 83	$\gg 0.03$
A40 $\times$ 2	0.0191	2 patients	≈ 166	$\gg 0.03$

Table 4.9: Cost breakdown the proxy uses at URGENT submission, with the corrected proxy semantics from `scheduler.cpp:proxy_function_min`. `vmCost@hzn` is GPU-rate $\times$ `first_finish_time` ≈ 125 s summed over the schedule’s scheduled jobs. `worstTardiCost` is the $100\times$ penalty applied to each unscheduled job’s worst-case lateness against the +30-minute patient deadline ($w_4=0.000396$ \$/s; `maxExecTime` on P600 $\times$ 1 = 3461 s). P600 $\times$ 2 wins because it keeps every runnable job scheduled and has the smallest URGENT worst-case among no-unscheduled-job plans.

Once URGENT is actually running on P600 $\times$ 2 it becomes part of `currentSched`. The next opt round at $t \approx 11.0$ sees JOB2 succeeded (its A40 $\times$ 1 slot freed) and `first_finish_time` now anchored on JOB1. Re-running the same proxy with URGENT already running on P600 $\times$ 2, migrating it to the now-free A40 $\times$ 1 is cheaper because URGENT’s projected completion collapses from 91.7 min to 13.9 min; in-proxy URGENT tardiness drops accordingly and the GPU-rate increase is two orders of magnitude smaller in dollars. Two rounds later, when JOB1 also completes at $t \approx 13.5$, the second A40 frees and URGENT upgrades to A40 $\times$ 2.

The literal answer to “why does not a very tardy job take the fastest bundle from the start?” is the following. Under `method=RG` the proxy sees only one event into the future, so URGENT’s intrinsic deadline-loss is invisible at the moment of placement. Meanwhile, any plan that drops a runnable patient — the only way to free an A40 slot — pays the $100\times$ `worstTardiCost` penalty. The optimiser’s choice is mechanically forced toward the slowest bundle that keeps everyone scheduled. The cluster converges to $A40\times 2$ through two or three migrations rather than a single decision. `method=PR` would pick $A40\times 2$ from the first round, but we keep `method=RG` as the reference deployment.

4.4 Scenario 2 — two types, forced 2-GPU placement

Scenario 2 introduces a second job type and constructs a workload that forces the optimiser to select a 2-GPU bundle for URGENT. Two priority-5 `cnn_big` pins occupy the A40 slots; an urgent `cnn_big` arrives with a deadline that admits $P600\times 2$ (20 min) but not $P600\times 1$ (34 min); the A40 slots are blocked by the pins.

Label	Type	Priority	Deadline	Epochs
PIN-A	B (<code>cnn_big</code>)	5	+10 min	80
PIN-B	B (<code>cnn_big</code>)	5	+10 min	80
P-LSTM-1	S (<code>lstm-small</code>)	1	+8 h	200
P-LSTM-2	S (<code>lstm-small</code>)	1	+8 h	200
URGENT*	B (<code>cnn_big</code>)	5	+22 min	40

Table 4.10: Scenario 2 submissions. PIN-A and PIN-B (tight `cnn_big`) are forced onto `polimi-gpu` $A40\times 1$. P-LSTM-1 and P-LSTM-2 (slack `lstm-small`) land on `matemagician` $P600\times 1$. URGENT’s deadline admits $P600\times 2$ (20 min) but not $P600\times 1$ (34 min); A40 is blocked by the priority-5 pins.

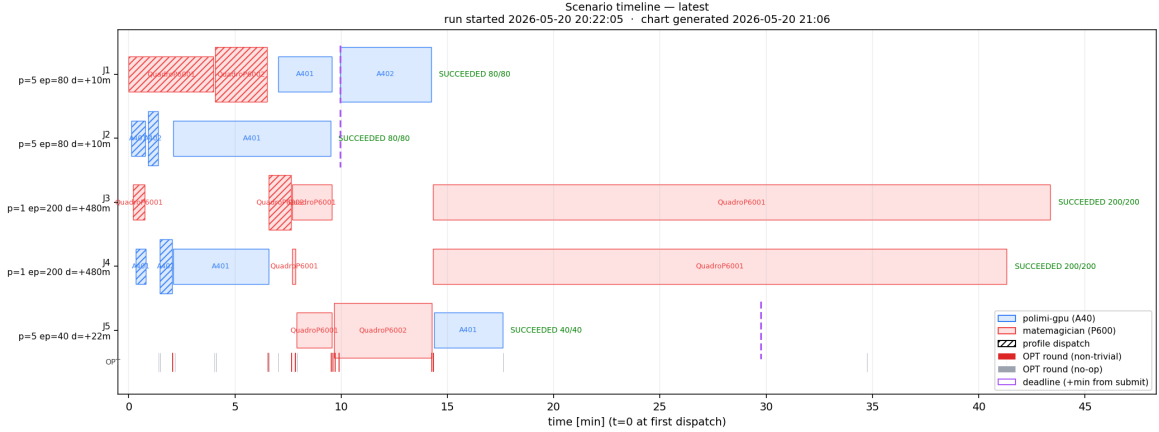


Figure 4.2: Scenario 2 timeline (run 2026-05-20, $t=0$ at 20:22:05 CEST; chart anchored at first dispatch). The profile sweep covers all eight cells (`cnn_big` $\times 4$, `lstm-small` $\times 4$) in ~ 8 min: PIN-A carries the `matemagician` P600 sweep, PIN-B the `polimi-gpu` A40 sweep, P-LSTM-1 and P-LSTM-2 atomic-claim the remaining cells. URGENT (`cnn_big` $p=5$, deadline +22 min, 40 ep) is submitted at $t\approx 7.5$ and placed on `matemagician` $P600\times 1$ (not $P600\times 2$), then upgrades to $P600\times 2$ at $t\approx 9$ once a patient `lstm-small` is displaced, and finally migrates to `polimi-gpu` $A40\times 1$ at $t\approx 15$ when the PINs vacate the A40 slots. URGENT completes at $t\approx 17.5$, well inside its +22-min deadline (purple dashed mark near min 30). All five jobs reach SUCCEEDED, zero FAILED.

4.4.1 Settled steady state (before URGENT)

PIN-A and PIN-B (`cnn_big`, prio 5, +10-min deadline) settle on `polimi-gpu` A40×1. P600×1 was rejected because its 68-min wall would overrun the 10-minute deadline. P-LSTM-1 and P-LSTM-2 (`lstm-small`, prio 1, +8-h deadline) settle on `matemagician` P600×1, the cheapest bundle for a slack `lstm-small`; the A40 slots are closed by the priority-5 pins.

4.4.2 URGENT placement and migrations

URGENT arrives at $t \approx 7.5$ min with deadline +22 min. Table 4.11 compares the candidate plans the proxy scored at submission.

Plan	ExecTime [s]	GPUCost [\$]	Tardiness [s]	TardCost [\$]	Total [\$]	Decision
A40×1 (preempt 1 pin)	252	0.0210	0	0.0	$\approx 1.57^\dagger$	rejected (unsched pin)
A40×2 (preempt both pins)	225	0.0344	0	0.0	$\approx 3.12^\dagger$	rejected (2 unsched pins)
P600×2 (preempt both lstm-small)	1 213	0.0185	0	0.0	≈ 0.019	not chosen (myopia + slack)
P600×1 (preempt 1 lstm-small)	2 042	0.0170	$0^\ddagger$	$0.0^\ddagger$	≈ 0.017	✓ chosen

Table 4.11: URGENT plan comparison. ExecTime = $40 \times$ per-epoch from Table 4.3; TardCost uses the priority-5 weight. † A40 plans look free of URGENT’s tardiness, but each evicted prio-5 pin becomes ~ 58 min tardy on its own +10-min deadline ($\approx \$1.55$ per victim), so they are dominated. ‡ In proxy at horizon, URGENT’s tardiness on P600×1 reads zero: the horizon (next-finish of a scheduled job) is well before URGENT’s +22-min deadline, so the proxy is blind to the 34-min real ExecTime. P600×1 wins on vmCost-at-horizon over P600×2, which is the horizon-myopic bug from §4.3.3 re-appearing in the lstm-small-eviction context.

4.4.3 Measured cascade

URGENT is placed on `matemagician` P600×1 in ~ 10 s at $t \approx 7.5$ min, preempting one lstm-small patient. About 1.5 min later, at $t \approx 9$ min, the optimiser upgrades URGENT to P600×2 (the second lstm-small is displaced) — the proxy re-evaluates with URGENT now in `currentSched`, sees its sluggish P600×1 finish time, and accepts the co-location. URGENT runs on P600×2 from $t \approx 9$ to $t \approx 15$. When PIN-B finishes at $t \approx 10$ and PIN-A at $t \approx 14$, the A40 slots free; the optimiser migrates URGENT to A40×1 at $t \approx 15$ (only ~ 2 min of work remains, so the $\times 2$ upgrade isn’t worth the kill-drain cost). URGENT completes at $t \approx 17.5$, well inside its +22-min deadline. The two lstm-small patients resume on the freed P600 slots and train through $t \approx 43$ min; one of them (P-LSTM-2) also briefly held an A40×1 slot during its A40 profile sweep before migrating back to P600. PIN-A and PIN-B are untouched throughout: their priority-5 plus tight-deadline status prices any A40-eviction cascade out of reach.

4.5 Implementation notes

The captured scenarios are reproducible only because of five fixes discovered while bringing them up. Each is documented in the same shape: symptom, root cause, fix, measured effect.

4.5.1 Slot-tracker race on quick stop/run pairs

Symptom. Under `method=PR`, a migration occasionally produced a `FAILED ec=1`. **Root cause.** A `/stop` and a `/run` issued to one worker within milliseconds arrived out of order, and the new container started before the previous CUDA context drained. **Fix.** `job_dispatcher._await_inflight_stops_on` makes every `dispatch_with_slot` await any in-flight `/stop` on the same node (bounded 30 s) before issuing `/run`.

4.5.2 Worker progress-lag and stop-drain stalls

Symptom. On `polimi-gpu`, the stdout reader fell progressively behind the trainer; an auto-preempt drain accumulated the backlog and overran the 60s drain-timeout safety net. **Root cause.** The reader (`worker/execution.py`) used to open a fresh PostgreSQL connection over the SSH-tunnelled link for every per-epoch `UPDATE jobs SET progress;` the TCP+SSH+psycopg handshake cost ~ 5 s per write, so lag grew by ~ 5 s per epoch. **Fix.** The reader now pushes only the latest progress into a shared slot; a coalescing background writer flushes via a short-lived connection at most once per second. **Effect.** Trainer-vs-worker timestamps match in real time, and the drain after a manual or auto-preempt completes in ~ 5 s rather than colliding with the 60 s ceiling.

4.5.3 Profile-claim lifecycle thrash

Symptom. Instances occasionally fell into a `claim`→`wipe`→`claim` loop that never converged. **Root cause.** Profile cell claims (rows in `profiling_results` with `duration_seconds` IS NULL) were garbage-collected by a hot-path sweep at the start of every scheduling round. A transient `is_profiling_run=FALSE` window during `_persist_assignment` was enough for the sweep to delete a live claim. **Fix.** The sweep is replaced by explicit cleanup at the life-cycle events that own the claim's fate: `DELETE /jobs` (single and bulk) drops the instance's unmeasured claims inline, and the worker drops them when a profile or standard run transitions to `FAILED` (`worker/execution.py`, `worker/reconcile.py`). A single long-lease garbage collector in the 60-minute `_queue_watcher` (`ProfilingScheduler.gc_orphaned_claims`) removes claims older than 15 min whose owner is no longer in `QUEUED` or `PROFILING` with `is_profiling_run=TRUE`. The hot path is now sweep-free.

A related fix changes the profile-placement view of `get_node_gpu_usage` to skip cross-instance in-flight claim reservations (`include_cross_instance_claims=False`). The standard-placement path still includes them; their original role of protecting a reserved cell from a competing standard run remains correct. Counting them for profile placement, however, mutually blocked peers competing for the same physical cell, producing the same thrash from a different angle. The node-slot semaphore is the real arbiter at dispatch time, so for profile placement only actually-running jobs are counted.

4.5.4 Reaper-wake gap after migrate-preempts

Symptom. A migrate plan that preempted `URGENT` via the planned channel occasionally left `URGENT` in `QUEUED` for ~ 23 minutes before the 60-minute watcher fired. **Root cause.** Splitting `ijm_slot_freed` into a terminal channel (which wakes the optimiser) and a planned channel (semaphore-release only) removed the original RG-cascade idle gap, but it did not cover the case where `_reset_stuck_queued_assignments` cleared the assignment and cancelled the dispatch task: no wake source remained. **Fix.** The reaper now sets `notify_event` whenever it actually reset at least one row (`backend/src/app.py`), so the next optimiser cycle re-replaces the orphan immediately.

4.5.5 Cross-version checkpoint compatibility

The captured Scenario 1 run exercises checkpoint compatibility in two directions. `JOB1` saves on `matemagician` (torch 1.5.1) and reloads on `polimi-gpu` (torch 2.6); the `URGENT` cascade victim `JOB4` saves on `polimi-gpu` and reloads on `matemagician`. Both rely on the legacy pre-1.6 `torch.save` format and `strict=False` load (`runtime/base.py`).

4.6 Reproducibility

Per-bundle costs are taken from `profiling_results.duration_seconds` (steady-state per-epoch mean, warmup excluded) and the hourly rates in Table 4.1. Timelines mirror per-instance state transitions from a 0.5 s API poller. Scenarios are driven by `infra/e2e_scenario.sh` and `infra/e2e_scenario_2types.sh`; slot metrics are exposed at `GET /admin/slots`, and API logs land at `/tmp/api.log` under `infra/launch.sh` tunnel.

Bibliography

- [AF⁺21] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.